KANT AX/AI 엔지니어 트랙 · 보충 학습 자료

리스트 컴프리헨션

바꾸고 고르는 for를
대괄호 한 줄로 쓰기

```
prices = [4500, 5000, 6500, 3000]

[p + 500 for p in prices]
# [5000, 5500, 7000, 3500]

[p for p in prices if p >= 5000]
# [5000, 6500]
```

오늘의 순서

1 왜 필요한가

길어지는 for와 읽기 어려운 map, 세 조각 옮기기, 가장 짧은 예

4–6쪽

2 만들기와 고르기

세 방법 비교, 읽는 순서, 뒤의 if, 고르고 바꾸기, 앞의 if else, 자리 비교

7–12쪽

3 여러 데이터에 쓰기

len과 range, 튜플 꺼내 담기, 딕셔너리와 집합 만들기

13–16쪽

4 함수와 함께 쓰기

sum, max에 넘기기, sorted와 min, for가 나을 때

17–19쪽

5 오류와 함정

뒤의 if에 else, print 넣기, 소괄호, 이름 어긋남

20–23쪽

6 실습

영수증, 품절 목록과 새 메뉴판, 성적표

24–26쪽

3쪽은 노트북 준비, 27–28쪽은 막힐 때와 한 장 요약이에요.

들어가기

22에서 이어서 해요

- ✓ 리스트, 튜플, for, append, join — 13
- ✓ 딕셔너리 items(), 집합 — 15, 16
- ✓ if, 한 줄 if else — 17
- ✓ range() — 18
- ✓ lambda, map, filter, sorted의 key — 21, 22

노트북 다시 열기

1. VS Code로 hello-uv 폴더 열기
2. 탐색기에서 `basics.ipynb` 열기
3. 오른쪽 위 커널이 .venv인지 보기
4. 셀에 적고 `Shift` + `Enter` 로 실행

기억이 안 나면 오른쪽에 적힌 챕터를 먼저 다시 봐요.

1. 왜 필요한가

for는 길고, map은 lambda까지 읽어야 해요

[1]
```python
prices = [4500, 5000, 6500]
up = []
for p in prices:
    up.append(p + 500)
print(up)
print(list(map(lambda p: p + 500, prices)))
```

네 줄

괄호가 겹쳐요

Python

```
[5000, 5500, 7000]
[5000, 5500, 7000]
```

for와 append

읽기는 쉬운데 빈 리스트, for, append 세 줄을 매번 적어요.

map과 lambda

한 줄이지만 list, map, lambda를 차례로 풀어 읽어야 해요.

한 줄로 쓰면서 for처럼 바로 읽히는 방법이 있어요.

1. 왜 필요한가

for의 세 조각을 대괄호 안에 옮겨 적어요

for로 쓰면

```
up = []
for p in prices:
    if p >= 5000:
        up.append(p + 500)
```

컴프리헨션으로 쓰면

```
up = [p + 500
      for p in prices
      if p >= 5000]
```

`p + 500` ① 넣을 값. append 괄호 안에 있던 것

`for p in prices` ② 하나씩 꺼내기. 콜론은 빼요

`if p >= 5000` ③ 고르기. 필요 없으면 빼요

1. 왜 필요한가

가장 짧은 컴프리헨션: 값마다 10 더하기

[2]
```python
numbers = [1, 2, 3]
print([x + 10 for x in numbers])
```

```
[11, 12, 13]
```

```
[x + 10 for x in numbers]
```



21과 같은 결과

list(map(lambda x: x + 10, numbers))와 같아요. list도 lambda도 없어요.

읽는 법

numbers에서 x를 하나씩 꺼내, x + 10을 모은 리스트예요.

[넣을 값 for 이름 in 리스트]는 값마다 계산한 새 리스트예요.

같은 일을 세 가지로 써요

[1]
```python
prices = [4500, 5000, 6500, 3000]
up = []                                          # 13의 for
for p in prices:
    up.append(p + 500)
print(up)
print(list(map(lambda p: p + 500, prices)))      # 21의 map
print([p + 500 for p in prices])                 # 컴프리헨션
```

```
[5000, 5500, 7000, 3500]
[5000, 5500, 7000, 3500]
[5000, 5500, 7000, 3500]
```

방법	줄 수	lambda
for	4줄	없어요
map	1줄	있어요
컴프리헨션	1줄	없어요

결과는 셋 다 같아요

새 리스트를 만들고 prices는 그대로예요. 짧고 바로 읽히는 컴프리헨션을 가장 많이 써요.

2. 만들기와 고르기

뒤의 for부터 읽고, 앞의 값을 모아요

```
[p + 500 for p in prices]
```

① `for p in prices` prices에서 p를 하나 꺼내요

② `p + 500` 꺼낸 p로 값을 계산해 모아요

꺼낸 p	p + 500	모인 리스트
4500	5000	[5000]
5000	5500	[5000, 5500]
6500	7000	[5000, 5500, 7000]
3000	3500	[5000, 5500, 7000, 3500]

2. 만들기와 고르기

뒤에 if를 붙이면 골라 담아요

[2]

```python
big = []                                                    17의 골라 담기
for p in prices:
    if p >= 5000:
        big.append(p)
print(big)
print(list(filter(lambda p: p >= 5000, prices)))    21의 filter
print([p for p in prices if p >= 5000])             if를 뒤에
```

Python

```
[5000, 6500]
[5000, 6500]
[5000, 6500]
```

```python
[p for p in prices if p >= 5000]
```

값은 p 그대로

바꾸지 않고 고르기만 해서 넣을 값이 p예요.

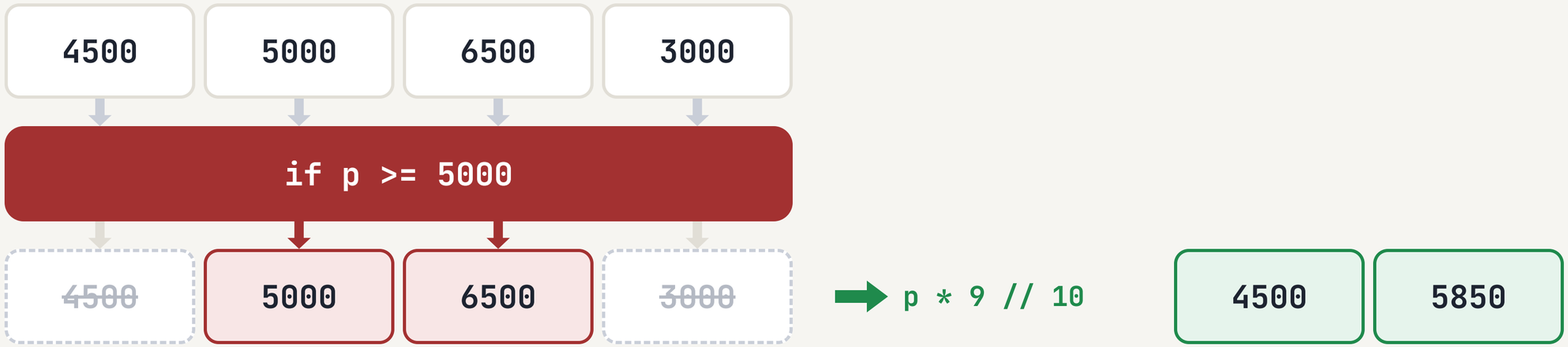
if가 True일 때만

if 뒤 조건이 True인 p만 모여요.

고르고 바꾸기를 한 번에 해요

[3]
```python
print([p * 9 // 10 for p in prices if p >= 5000])
print(list(map(lambda p: p * 9 // 10, filter(lambda p: p >= 5000, prices))))
```
컴프리헨션
21의 겹쳐 쓰기
Python

```
[4500, 5850]
[4500, 5850]
```



if로 고른 값만 앞의 식으로 바꿔서 모아요.

2. 만들기와 고르기

앞에 한 줄 if else를 넣으면 값을 바꿔 적어요

[4]
```python
print(["비쌈" if p >= 5000 else "보통" for p in prices])
```
Python

```
['보통', '비쌈', '비쌈', '보통']
```

```python
["비쌈" if p >= 5000 else "보통" for p in prices]
```

p	p >= 5000	넣을 값
4500	False	"보통"
5000	True	"비쌈"
6500	True	"비쌈"
3000	False	"보통"

17의 한 줄 if else

값1 if 조건 else 값2는 값 하나라서 넣을 값 자리에 들어가요.

2. 만들기와 고르기

if가 뒤에 있으면 고르기, 앞에 있으면 바꾸기

[5]
```python
print([p for p in prices if p >= 5000])
print([p if p >= 5000 else 0 for p in prices])
```

```
[5000, 6500]
[0, 5000, 6500, 0]
```

자리로 구분해요

for보다 뒤인지 앞인지 봐요.

뒤의 if 고르기

```
[p for p in prices if p >= 5000]
# [5000, 6500]
```

앞의 if else 바꾸기

```
[p if p >= 5000 else 0 for p in prices]
# [0, 5000, 6500, 0]
```

조건에 맞는 값만 남아요

else를 쓰지 않고, 개수가 줄어요.

모든 값을 바꿔 적어요

else가 꼭 있어야 하고, 개수가 그대로예요.

3. 여러 데이터에 쓰기

길이, 글자, range로도 만들어요

[1]
```python
names = ["latte", "tea", "coffee"]
print([len(m) for m in names])
print([m[0] for m in names])
print([n * n for n in range(1, 6)])
```

```
[5, 3, 6]
['l', 't', 'c']
[1, 4, 9, 16, 25]
```

in 뒤에는 for에 쓰던 것

리스트, 글자, range처럼 for로 돌 수 있는 건 모두 넣을 수 있어요.

`[len(m) for m in names]` **`[5, 3, 6]`** 함수를 불러도 돼요. 21의 map(len, names)와 같아요

`[m[0] for m in names]` **`['l', 't', 'c']`** 13의 글자 꺼내기. 첫 글자만 모아요

`[n * n for n in range(1, 6)]` **`[1, 4, 9, …]`** 18의 range로 숫자를 만들며 계산해요

3. 여러 데이터에 쓰기

튜플 리스트는 꺼내 담기로 이름을 붙여요

```
[2] orders = [("kim", "latte", 2), ("lee", "cake", 1), ("park", "coffee", 3), ("kim", "tea", 1)]
    print([o[0] for o in orders])                                          번호로 꺼내기
    print([menu_name for name, menu_name, qty in orders if qty >= 2])      이름 붙여 꺼내기
                                                                           Python
['kim', 'lee', 'park', 'kim']
['latte', 'coffee']
```

튜플 하나 → `"kim"` `"latte"` `2` → `name` `menu_name` `qty`

13의 for name, menu_name, qty in

for에서 쓰던 꺼내 담기를 그대로 써요. 번호 대신 이름으로 읽혀요.

21의 filter와 map을 한 줄에

2잔 이상 주문만 고르고, 메뉴 이름만 꺼냈어요.

3. 여러 데이터에 쓰기

중괄호와 키: 값으로 딕셔너리도 만들어요

[3]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
print({m: p * 11 // 10 for m, p in menu.items()})
stock = {"coffee": 3, "latte": 0, "cake": 2}
print({m: n for m, n in stock.items() if n > 0})
print([m for m in stock if stock[m] == 0])
```
Python

```
{'coffee': 4950, 'latte': 5500, 'cake': 7150}
{'coffee': 3, 'cake': 2}
['latte']
```

```python
{m: p * 11 // 10 for m, p in menu.items()}
```

키: 값을 앞에 적어요

15의 items()로 키와 값을 꺼내고, 새 키와 새 값을 콜론으로 이어요.

키만 필요하면 리스트

품절 메뉴 이름만 필요하면 대괄호로 리스트를 만들어요.

{키: 값 for ...}는 딕셔너리, [값 for ...]는 리스트예요.

3. 여러 데이터에 쓰기

중괄호에 값만 적으면 집합이 돼요

[4]
```python
buyers = {o[0] for o in orders}
print(len(buyers))
print(sorted(buyers))
```

```
3
['kim', 'lee', 'park']
```

16의 집합 규칙 그대로

kim이 두 번 주문했지만 한 번만 남아요.



찍는 순서는 그때그때 달라요

집합은 순서가 없어서 sorted로 줄 세워 찍었어요.

괄호 모양으로 구분해요

[값]은 리스트, {값}은 집합, {키: 값}은 딕셔너리예요.

4. 함수와 함께 쓰기

sum, max에 바로 넘겨요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
cart = ["coffee", "cake", "latte", "latte"]
print(sum([menu[m] for m in cart]))
print(max([len(m) for m in menu]))
```
장바구니 합계

Python

```
21000
6
```

19의 calc_total 몸통 세 줄

```
total = 0
for m in cart:
    total += menu[m]
```

```
21000
```

가격 리스트를 만들어 더해요

cart의 메뉴마다 menu[m]으로 가격을 꺼내 리스트를 만들고, sum이 그 리스트를 더해요. 메뉴 이름 중 가장 긴 길이는 6이에요.

계산할 값만 리스트로 모아 sum, max, len에 넘겨요.

고른 다음 22의 sorted, min으로 이어요

[2]
```python
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 5}
ok = [m for m in menu if stock[m] > 0]
print(ok)
print(sorted(ok, key=menu.get))
print(min(ok, key=menu.get))
```
주문 가능한 메뉴

Python

```
['coffee', 'cake', 'tea']
['tea', 'coffee', 'cake']
tea
```

고르기는 컴프리헨션

품절이 아닌 메뉴만 골라 ok에 담아요.

줄 세우기는 sorted

22처럼 key=menu.get으로 가격순이에요. 주문 가능한 가장 싼 메뉴는 tea예요.

고르고 바꾸기는 컴프리헨션, 줄 세우기와 고르기 하나는 sorted, min이에요.

4. 함수와 함께 쓰기

이럴 땐 for가 더 읽기 쉬워요

새 리스트 하나를 만들 때	**컴프리헨션**	`[p + 500 for p in prices]`
print로 찍기만 할 때	**for**	`for m in names: print(m)`
한 번에 여러 줄을 할 때	**for**	합계 쌓기, 재고 줄이기
break로 멈춰야 할 때	**for**	18의 break
조건이 두 개를 넘을 때	**for나 def**	if, elif로 나눠 쓰기

참고, 읽기만: 겹친 컴프리헨션

[3]
```python
scores = [[90, 80], [70, 60]]
print([x for row in scores
       for x in row])
```

```
[90, 80, 70, 60]
```

for를 두 번 적을 수도 있어요

14의 2차원 리스트를 한 줄로 펼쳤어요. 앞의 for가 바깥 반복이에요. 이런 코드를 만나면 읽을 수 있게만 봐 두세요.

5. 오류와 함정

뒤의 if에 else를 붙이면 SyntaxError가 나요

[1]
```
prices = [4500, 5000, 6500, 3000]
print([p for p in prices if p >= 5000 else 0])  뒤에는 else 없음
```
Python

```
SyntaxError: invalid syntax
```

이렇게 고쳐요 고르기 또는 바꾸기

```
[p for p in prices if p >= 5000]
[p if p >= 5000 else 0 for p in prices]
```

오류 끝 괄호는 뺐어요

SyntaxError 끝의 (1234.py, line 2)는 셀마다 달라서 슬라이드에서 뺐어요.

else는 앞에서만

for 뒤의 if는 고르기라서 else를 받지 않아요. 바꾸려면 if else를 앞으로 옮겨요.

셀 전체가 실행되지 않아요

SyntaxError는 실행 전에 나서 첫 줄 prices도 만들어지지 않아요.

5. 오류와 함정

print를 넣으면 None 리스트가 생겨요

[2]
```
[print(m) for m in ["latte", "tea"]]
```
print가 돌려준 None

Python

```
latte
tea


[None, None]
```

찍히긴 하는데

print는 화면에 찍고 None을 돌려줘요. 19에서 본 그 None이 리스트에 모였어요.

컴프리헨션은 값을 모으는 곳

결과 리스트가 필요 없는 일, 곧 찍기만 하는 일에는 쓰지 않아요. 셀 마지막 줄이라 [None, None]이 결과로 보였어요.

이렇게 고쳐요 찍기는 for

```
for m in ["latte", "tea"]:
    print(m)
```

5. 오류와 함정

소괄호로 감싸면 리스트가 아니에요

[3]
```python
prices = [4500, 5000, 6500, 3000]
print((p + 500 for p in prices))
print(list(p + 500 for p in prices))
```
튜플이 아니에요

Python

```
<generator object <genexpr> at 0x7f2ba871b440>
[5000, 5500, 7000, 3500]
```

21의 map object와 비슷해요

소괄호는 달라고 하면 값을 하나씩 꺼내 주는 묶음을 만들어요. list로 감싸야 결과가 보여요. 튜플을 만들지 않아요.

리스트가 필요하면 대괄호

[p + 500 for p in prices]처럼 대괄호로 써요. 0x로 시작하는 숫자는 화면마다 달라요.

5. 오류와 함정

앞과 뒤의 이름이 다르면 NameError가 나요

[4]
```
print([x + 10 for p in prices])
```
x와 p Python

```
NameError: name 'x' is not defined
```

```
[x + 10 for p in prices]
```

복사해 고칠 때 자주 나요

앞 예제의 [x + 10 for x in numbers]를 가져와 뒤만 바꾸면 생겨요. 앞과 뒤에서 같은 이름을 쓰는지 봐요.

꺼낸 이름은 p예요

for p in으로 꺼냈는데 앞에서 x를 썼어요. 그런 변수는 없어요.

이렇게 고쳐요 이름 맞추기

```
[p + 10 for p in prices]
```

6. 실습

장바구니 영수증을 컴프리헨션으로 찍어요

[1]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500}
cart = ["latte", "cake", "coffee"]
lines = [f"{m:<7}{menu[m]:>6,}원" for m in cart]
print("\n".join(lines))
print(f"합계   {sum([menu[m] for m in cart]):>6,}원")
```

줄 만들기

합계

Python

```
latte   5,000원
cake    6,500원
coffee  4,500원
합계   16,000원
```

f-string도 값이에요

13의 칸 맞추기를 넣을 값 자리에 그대로 적었어요.

21에서는 def와 map

줄 만드는 일을 def로 따로 만들었어요. 이번에는 한 줄이에요.

줄 리스트를 만들고 join으로 이어 찍어요.

6. 실습

품절 목록과 주문 가능 메뉴판을 만들어요

[2]
```python
menu = {"coffee": 4500, "latte": 5000, "cake": 6500, "tea": 4000}
stock = {"coffee": 3, "latte": 0, "cake": 2, "tea": 0}
soldout = [m for m in stock if stock[m] == 0]
print("품절:", ", ".join(soldout))
board = {m: menu[m] * 11 // 10 for m in menu if stock[m] > 0}
print(board)
```

리스트

딕셔너리

Python

```
품절: latte, tea
{'coffee': 4950, 'cake': 7150}
```

coffee	cake
4950	7150

고르고 바꾸기

재고 있는 메뉴만 골라 10% 올린 가격을 담아요.

join으로 한 줄

품절 메뉴를 쉼표로 이어 찍어요.

6. 실습

성적표에서 합격자, 결과표, 평균, 1등을 뽑아요

[3]
```python
scores = [["kim", 92], ["lee", 78], ["park", 85], ["choi", 59]]
print([name for name, score in scores if score >= 80])
print(["합격" if score >= 60 else "재시험" for name, score in scores])
print(sum([score for name, score in scores]) / len(scores))
print(max(scores, key=lambda s: s[1])[0])
```

Python

```
['kim', 'park']
['합격', '합격', '합격', '재시험']
78.5
kim
```

14의 2차원 리스트

안쪽 리스트 두 칸을 name, score로 꺼내 담아요.

if 뒤에 80점 이상만 고르기

if else 앞에 모두에게 합격, 재시험 붙이기

sum([...]) / len 점수만 모아 평균

max(..., key=) 22의 key로 1등

정리

막히면 오류의 마지막 줄을 봐요

1	`SyntaxError: invalid syntax` 뒤의 if에 else를 붙였어요	else는 for 앞으로
2	`NameError: name 'x' is not defined` 앞과 뒤의 이름이 달라요	같은 이름으로
3	[None, None]이 같이 나와요 대괄호 안에서 print를 했어요	찍기는 for로
4	<generator object ...>가 나와요 소괄호로 감쌌어요	대괄호로
5	개수가 줄었어요 for 뒤에 if를 붙여 골랐어요	바꾸려면 if else를 앞에
6	한 줄이 너무 길어요 조건이나 할 일이 많아요	for나 def로 나누기
7	집합 순서가 슬라이드와 달라요 집합은 순서가 없어요	sorted()로 찍기

헷갈리면 같은 일을 for 문으로 먼저 써 보고 세 조각을 옮겨요.

한 장 요약

`[x + 10 for x in xs]`

값마다 바꾼 리스트

`[x for x in xs if x > 0]`

조건에 맞는 값만

`[a if c else b for x in xs]`

앞의 if else는 바꾸기

`[n for n, s in pairs]`

튜플은 꺼내 담기로

`{k: v for k, v in d.items()}`

딕셔너리 만들기

`{x for x in xs}`

집합 만들기

`sum([...])`

모은 값을 바로 넘기기

for 문

찍기, 여러 줄, break

뒤의 if는 고르기, 앞의 if else는 바꾸기

다음 단계: 함수, 정렬, 컴프리헨션을 섞어 카페 데이터를 다뤄요